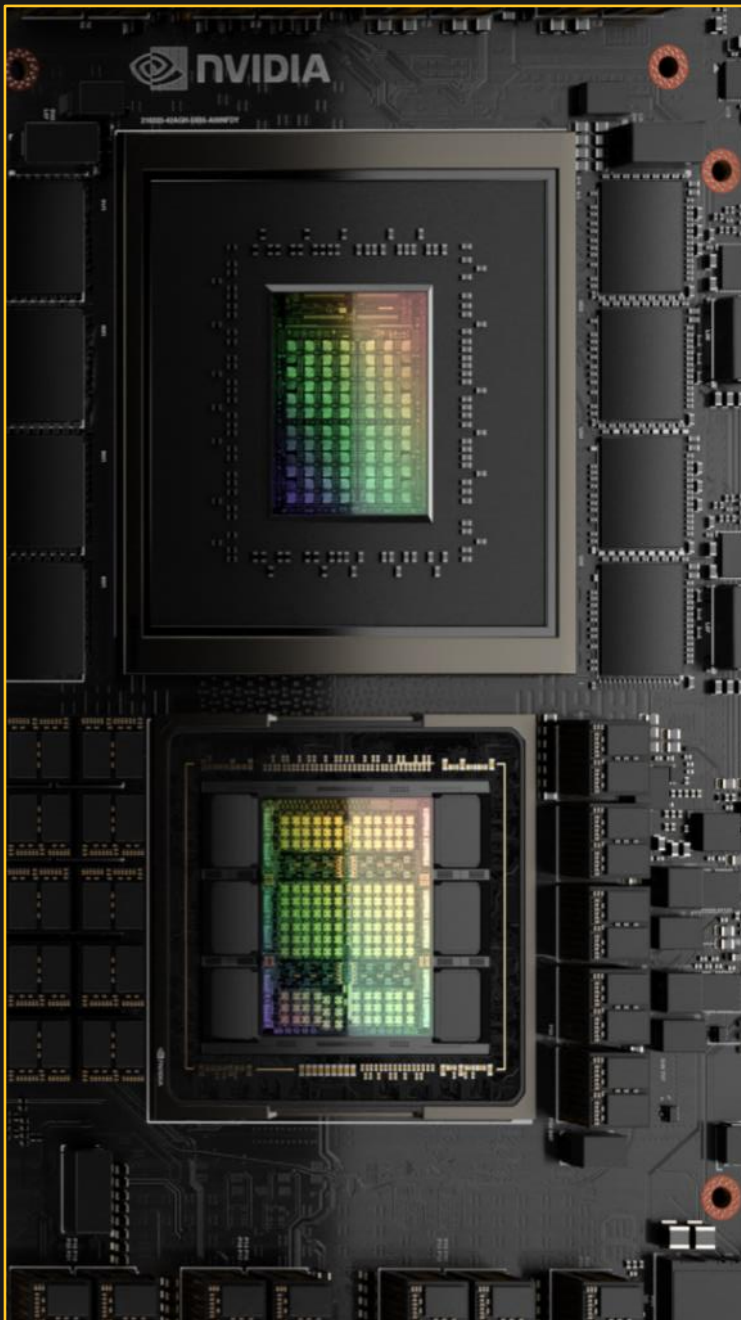


WUG 2026-07-08: Grace Hoppers and Multi-Arch Clusters with Warewulf

Josh Burks
Senior HPC Systems Analyst
Research Technology Office
Arizona State University

Presented at WUG, July 08, 2026



A Brief Overview Of Grace Hoppers

- **Grace CPU**
 - 72x Arm Neoverse V2
 - 512 GB LPDDR5X system memory
 - Linux aarch64 userland
- **Hopper GPU**
 - H100-class GPU
 - 96GB HBM3 GPU memory
 - CUDA acceleration
- **NVLink-C2C**
 - Coherent CPU-GPU link
 - 900 GB/s total bandwidth, 450 GB/s/dir.

What makes Grace Hopper different

Aarch64, not x86_64

The OS, kernel, initramfs, userland RPMs, libraries, and application builds must be Arm-native.

Unified memory

CPU and GPU can share system-allocated memory through shared page tables and coherent interconnect behavior.

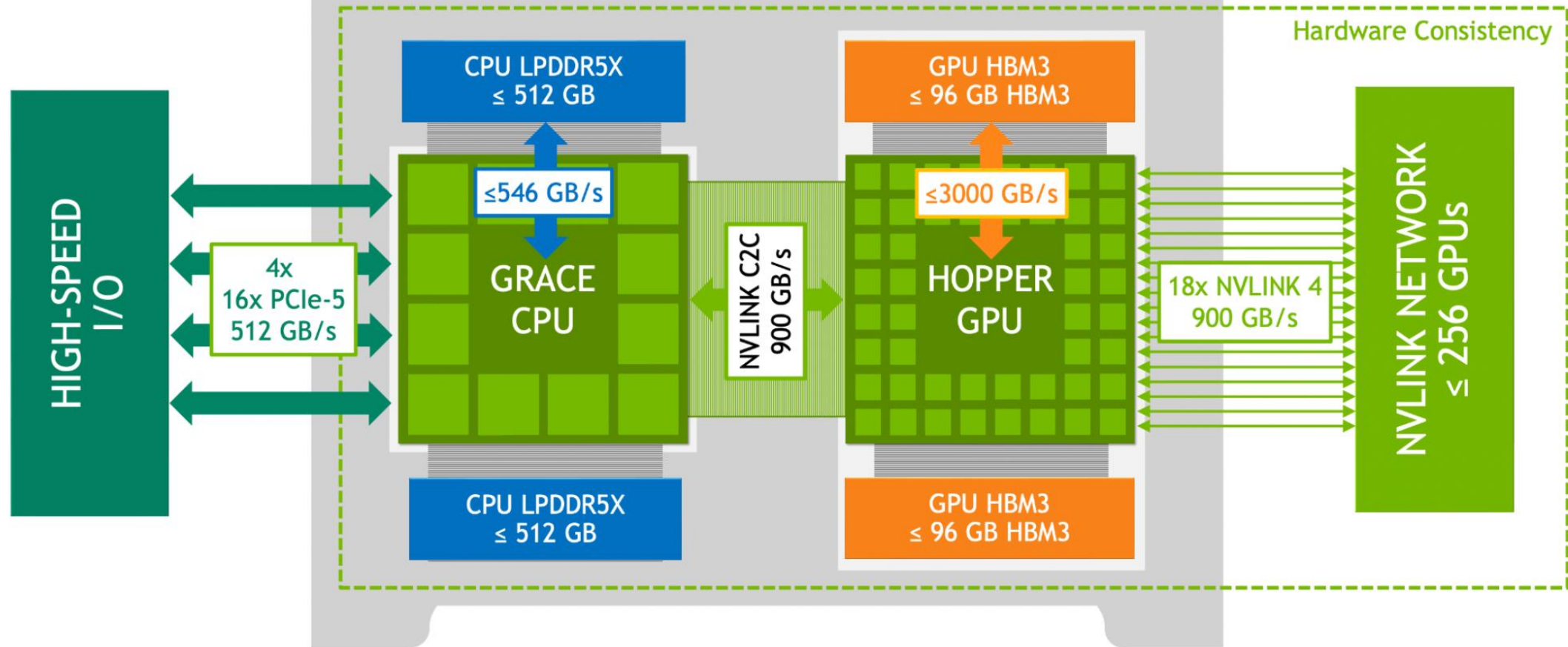
Varying software support

Some packages, wheels, or vendor tools are solely for x86_64

Operational visibility

Users should know they are targeting an Arm GPU node before a job starts

NVIDIA Grace Hopper Superchip



TOP500 Release

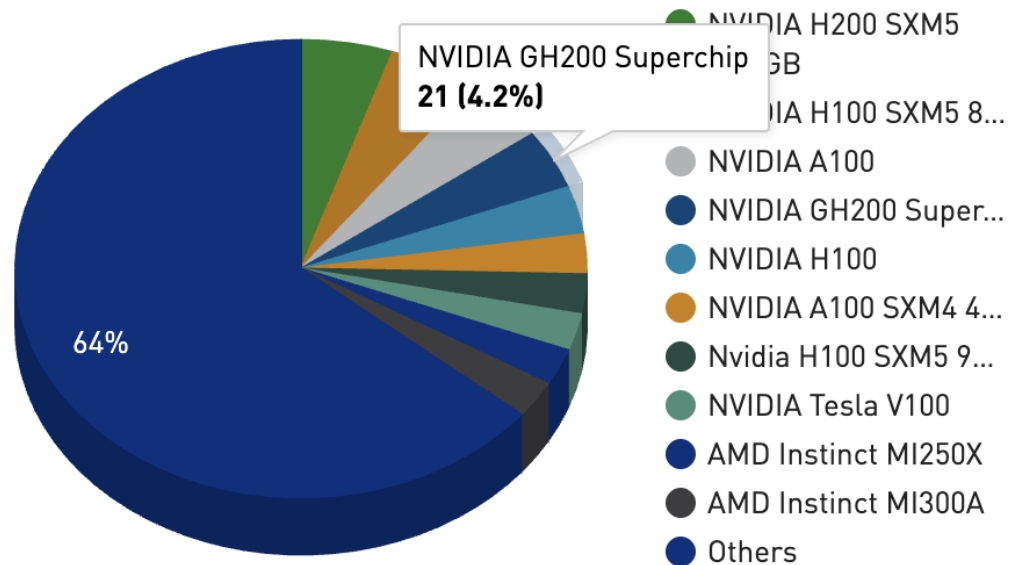
June 2026

Category

Accelerator/Co-Processor

Submit

Accelerator/Co-Processor System Share



Unified Memory & Stateless Provisioning

The tension

Warewulf commonly runs nodes with the operating system in memory.
On GH200, memory is a first-class compute resource for GPU workflows.

The risk

A large in-memory rootfs can consume memory that users expect to be available for CPU-GPU workloads.

The practical approach

Move the rootfs to local disk, keep memory available for GPU

AArch64 node images

Separate image required

Grace Hopper nodes need an AArch64 rootfs.

Common situation

The Warewulf controller is x86_64.

The GH200 image is AArch64.

A Translation layer is needed

binfmt_misc and qemu

```
[root@warewulf:~]# wwctl image shell sol-aarch64-rocky10.1-cuda-595.71.05.20260706
```

```
Image build will be skipped if the shell ends with a non-zero exit code.
```

```
ERROR: exec format error
```

```
ERROR: command returned an error: [/bin/bash]: exit status 255
```

```
[root@warewulf:~]# file $(wwctl image show sol-aarch64-rocky10.1-cuda-595.71.05.20260706)/usr/bin/bash
/var/lib/warewulf/chroots/sol-aarch64-rocky10.1-cuda-595.71.05.20260706/rootfs/usr/bin/bash: ELF 64-bit
LSB shared object, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-aarch6
4.so.1, BuildID[sha1]=56bcb088cc278c3fc8798d5aad89b8f7ca70295b, for GNU/Linux 3.7.0, stripped
```

```
[root@warewulf:~]# █
```

Translation Layers

binfmt_misc

Miscellaneous Binary Format

qemu

Quick Emulator's Tiny Code Generator

Translation Layer

☰ QEMU

🌐 23 languages ▾

Article [Talk](#)

[Read](#) [Edit](#) [View history](#) ⋮

From Wikipedia, the free encyclopedia

The **Quick Emulator (QEMU)**^[3] is a free and open-source machine emulator and virtualizer. As a Virtual Machine Monitor (VMM) it supports a number of hypervisors, including the Linux-based Kernel-based Virtual Machine (KVM), Xen, MacOS's HVF, Windows' Hyper-V and a number of others.^[4] It is also capable of emulating a number of instruction set architectures on any supported host through its JIT binary translator known as the Tiny Code Generator (TCG). This allows it to emulate full systems or run individual programs compiled for one processor architecture on any other. QEMU supports the emulation of x86, ARM, PowerPC, RISC-V, and many others.^[5]

Operating modes [\[edit \]](#)

QEMU has multiple operating modes:^[8]

- **User-mode emulation.** In the user emulation mode, QEMU runs single Linux or Darwin/macOS programs that were compiled for a different instruction set. System calls are thunked for endianness and for 32/64-bit mismatches. Fast cross-compilation and cross-debugging are the main targets for user-mode emulation.

Translation Layers

binfmt_misc

🌐 2 languages ▾

Article [Talk](#)

Read [Edit](#) [View history](#) ⋮

From Wikipedia, the free encyclopedia

binfmt_misc (*Miscellaneous Binary Format*) is a capability of the [Linux kernel](#) which allows arbitrary [executable file formats](#) to be recognized and passed to certain [user space](#) applications, such as [emulators](#) and [virtual machines](#).^[1] It is one of a number of binary format handlers in the kernel that are involved in preparing a user-space program to run.^[2]

Translation Layers

binfmt_misc

🌐 2 languages ▾

Common usage [\[edit \]](#)

istory ⋮

- binfmt_misc allows [Java](#) programs to be passed directly to the [Java virtual machine](#).^[6]
- binfmt_misc allows identification of [PE executables](#) using the magic number "MZ". Examples:
 - `:DOSWin:M::MZ::/usr/bin/wine:` will hand all such files to [Wine](#) (thus assuming they are ordinary [MS-DOS](#) or [Microsoft Windows](#) executable)
 - `:CLR:M::MZ::/usr/bin/mono:` will hand such files to [Mono](#) (thus assuming they are [.NET](#) executables)^{[7][8]}
 - Microsoft's [WSL1](#) uses binfmt to allow for calling Windows programs from Linux. The line is equivalent to `:WSLInterOP:M::MZ::/init:P.`

kernel
ed to
[1] It is
in

qemu-*-static and syscall forwarding

Guest instruction example

```
add x0, x1, x2  
str x0, [sp, #16]  
bl  some_function
```

QEMU behavior

Guest registers like x0, x1, sp, and pc are represented in QEMU CPU state.
Translated host code updates that emulated CPU state.

Syscall behavior

AArch64 Linux uses svc #0 to enter the kernel.

QEMU catches the guest syscall.

QEMU maps the AArch64 syscall ABI to the host Linux syscall ABI.

The host kernel performs the real file, process, memory, or network operation.

Installing QEMU and binfmt_misc

- **Docker**

```
docker run --rm --privileged multiarch/qemu-user-static --reset -p yes
```

- **Podman**

```
podman run --rm --privileged multiarch/qemu-user-static --reset -p yes
```

- **Apptainer**

```
apptainer run docker://multiarch/qemu-user-static --reset -p yes
```

```
[root@warewolf:~]# ls /proc/sys/fs/binfmt_misc/qemu-*
/proc/sys/fs/binfmt_misc/qemu-aarch64          /proc/sys/fs/binfmt_misc/qemu-mipsn32el
/proc/sys/fs/binfmt_misc/qemu-aarch64_be       /proc/sys/fs/binfmt_misc/qemu-or1k
/proc/sys/fs/binfmt_misc/qemu-alpha            /proc/sys/fs/binfmt_misc/qemu-ppc
/proc/sys/fs/binfmt_misc/qemu-arm              /proc/sys/fs/binfmt_misc/qemu-ppc64
/proc/sys/fs/binfmt_misc/qemu-armeb            /proc/sys/fs/binfmt_misc/qemu-ppc64le
/proc/sys/fs/binfmt_misc/qemu-hexagon           /proc/sys/fs/binfmt_misc/qemu-riscv32
/proc/sys/fs/binfmt_misc/qemu-hppa             /proc/sys/fs/binfmt_misc/qemu-riscv64
/proc/sys/fs/binfmt_misc/qemu-m68k             /proc/sys/fs/binfmt_misc/qemu-s390x
/proc/sys/fs/binfmt_misc/qemu-microblaze        /proc/sys/fs/binfmt_misc/qemu-sh4
/proc/sys/fs/binfmt_misc/qemu-microblazeel      /proc/sys/fs/binfmt_misc/qemu-sh4eb
/proc/sys/fs/binfmt_misc/qemu-mips             /proc/sys/fs/binfmt_misc/qemu-sparc
/proc/sys/fs/binfmt_misc/qemu-mips64           /proc/sys/fs/binfmt_misc/qemu-sparc32plus
/proc/sys/fs/binfmt_misc/qemu-mips64el         /proc/sys/fs/binfmt_misc/qemu-sparc64
/proc/sys/fs/binfmt_misc/qemu-mipsel           /proc/sys/fs/binfmt_misc/qemu-xtensa
/proc/sys/fs/binfmt_misc/qemu-mipsn32          /proc/sys/fs/binfmt_misc/qemu-xtensaeb
```

```
[root@warewolf:~]# cat /proc/sys/fs/binfmt_misc/qemu-aarch64
```

```
enabled
interpreter /usr/bin/qemu-aarch64-static
flags: F
offset 0
magic 7f454c460201010000000000000000000000000200b700
mask ffffffffffffffffff00ffffffffffffffffffffffffff
```

```
[root@warewolf:~]#
```

```
[root@warewulf:~]# uname -m
x86_64
[root@warewulf:~]# podman run --rm --privileged multiarch/qemu-user-static --reset -p yes
Setting /usr/bin/qemu-alpha-static as binfmt interpreter for alpha
Setting /usr/bin/qemu-arm-static as binfmt interpreter for arm
Setting /usr/bin/qemu-armeb-static as binfmt interpreter for armb
Setting /usr/bin/qemu-sparc-static as binfmt interpreter for sparc
Setting /usr/bin/qemu-sparc32plus-static as binfmt interpreter for sparc32plus
Setting /usr/bin/qemu-sparc64-static as binfmt interpreter for sparc64
Setting /usr/bin/qemu-ppc-static as binfmt interpreter for ppc
Setting /usr/bin/qemu-ppc64-static as binfmt interpreter for ppc64
Setting /usr/bin/qemu-ppc64le-static as binfmt interpreter for ppc64le
Setting /usr/bin/qemu-m68k-static as binfmt interpreter for m68k
Setting /usr/bin/qemu-mips-static as binfmt interpreter for mips
Setting /usr/bin/qemu-mipsel-static as binfmt interpreter for mipsel
Setting /usr/bin/qemu-mipsn32-static as binfmt interpreter for mipsn32
Setting /usr/bin/qemu-mipsn32el-static as binfmt interpreter for mipsn32el
Setting /usr/bin/qemu-mips64-static as binfmt interpreter for mips64
Setting /usr/bin/qemu-mips64el-static as binfmt interpreter for mips64el
Setting /usr/bin/qemu-sh4-static as binfmt interpreter for sh4
Setting /usr/bin/qemu-sh4eb-static as binfmt interpreter for sh4eb
Setting /usr/bin/qemu-s390x-static as binfmt interpreter for s390x
Setting /usr/bin/qemu-aarch64-static as binfmt interpreter for aarch64
Setting /usr/bin/qemu-aarch64_be-static as binfmt interpreter for aarch64_be
Setting /usr/bin/qemu-hppa-static as binfmt interpreter for hppa
Setting /usr/bin/qemu-riscv32-static as binfmt interpreter for riscv32
Setting /usr/bin/qemu-riscv64-static as binfmt interpreter for riscv64
Setting /usr/bin/qemu-xtensa-static as binfmt interpreter for xtensa
Setting /usr/bin/qemu-xtensaeb-static as binfmt interpreter for xtensaeb
Setting /usr/bin/qemu-microblaze-static as binfmt interpreter for microblaze
Setting /usr/bin/qemu-microblazeel-static as binfmt interpreter for microblazeel
Setting /usr/bin/qemu-or1k-static as binfmt interpreter for or1k
Setting /usr/bin/qemu-hexagon-static as binfmt interpreter for hexagon
[root@warewulf:~]# wwctl image shell sol-aarch64-rocky10.1-cuda-595.71.05.20260706
Image build will be skipped if the shell ends with a non-zero exit code.
[ sol-aarch64-rocky10.1-cuda-595.71.05 ~]# uname -m
aarch64
[ sol-aarch64-rocky10.1-cuda-595.71.05 ~]#
```

Maintaining Consistency

With multiple node images for one cluster, consistency matters.

Keep aligned where possible

- OS release
- Site packages
- NVIDIA driver branch
- OFED or NIC driver stack
- Monitoring and security agents

Automate the build

- Containerfiles
- Ansible
- Makefile
- Bash pipeline
- CI job

Maintaining Consistent Packages

ASU's approach: one large file list of packages, passed into every Container build

```
[root@warewolf:rc-warewolf-container-builds]# grep -A1 "# Install from the list of packages" Containerfile.gracehopper Containerfile.solcuda
Containerfile.gracehopper:# Install from the list of packages
Containerfile.gracehopper-RUN dnf -yq install $(< /mnt/packages) --nobest --skip-broken
--
Containerfile.solcuda:# Install from the list of packages
Containerfile.solcuda-RUN dnf -yq install $(< /mnt/packages) --nobest --skip-broken
[root@warewolf:rc-warewolf-container-builds]# head packages
ImageMagick
ImageMagick-c++-devel
Lmod
VirtualGL
Xvfb
alloy
apptainer
atlas-devel
attr
audit
[root@warewolf:rc-warewolf-container-builds]#
```

Maintaining Consistent Drivers

ASU's approach: Passed variables into every Container build, pre-stage Nvidia runfiles

```
[root@warewulf:rc-warewulf-container-builds]# grep -A9 "# Install NVIDIA Drivers" Containerfile.gracehopper Containerfile.solcuda
Containerfile.gracehopper:# Install NVIDIA Drivers
Containerfile.gracehopper-ARG NVIDIA_VERSION
Containerfile.gracehopper-RUN cd /mnt/${BUILD_TMP_DIR} && chmod +x NVIDIA-Linux-${TARGET_ARCH}-${NVIDIA_VERSION}.run && \
Containerfile.gracehopper-    /mnt/${BUILD_TMP_DIR}/NVIDIA-Linux-${TARGET_ARCH}-${NVIDIA_VERSION}.run \
Containerfile.gracehopper-    --silent \
Containerfile.gracehopper-    --kernel-source-path=/usr/src/kernels/$(rpm -q kernel-64k-core --qf '%{version}-%{release}.%{arch}+64k') \
Containerfile.gracehopper-    --kernel-name=$(rpm -q kernel-64k-core --qf '%{version}-%{release}.%{arch}+64k') \
Containerfile.gracehopper-    --systemd \
Containerfile.gracehopper-    -m=kernel-open
Containerfile.gracehopper-
Containerfile.solcuda:# Install NVIDIA Drivers
Containerfile.solcuda-ARG NVIDIA_VERSION
Containerfile.solcuda-RUN cd /mnt/${BUILD_TMP_DIR} && chmod +x NVIDIA-Linux-${TARGET_ARCH}-${NVIDIA_VERSION}.run && \
Containerfile.solcuda-    /mnt/${BUILD_TMP_DIR}/NVIDIA-Linux-${TARGET_ARCH}-${NVIDIA_VERSION}.run \
Containerfile.solcuda-    --silent \
Containerfile.solcuda-    --kernel-source-path=/usr/src/kernels/$(rpm -q kernel-core --qf '%{version}-%{release}.%{arch}\n' | tail -n 1) \
Containerfile.solcuda-    --kernel-name=$(rpm -q kernel-core --qf '%{version}-%{release}.%{arch}\n' | tail -n 1) \
Containerfile.solcuda-    --systemd \
Containerfile.solcuda-    -m=kernel-open
Containerfile.solcuda-
[root@warewulf:rc-warewulf-container-builds]#
```

Maintaining Consistency

```
# Generic function for podman build
# podman_build parameters:
# $(1): Target/variant name (cuda, rocm, etc.)
# $(2): Container tag name (sol-x86_64-rocky8.10-cuda-555.42.02, etc.)
# $(3): Architecture (x86_64, aarch64)
# $(4): OS version (8.10, 9.5, etc.)
# $(5): Repository overaly name (repos, repos-aarch64, etc.)
define podman_build
    @echo "$(GREEN)Building $(BLUE)$(1)$(GREEN) variant$(NC)"
    $(call prepare_repos,$(5))
    if [ "$(shell uname -m)" != "$(3)" ]; then \
        echo "$(YELLOW)Setting up QEMU for cross-architecture build ($(BLUE)$(3)$(YELLOW))...$(NC)"; \
        podman run --rm --privileged multiarch/qemu-user-static --reset -p yes; \
    fi
    @podman build $(PODMAN_ARGS) \
        --file ./Containerfile.$(1) \
        --build-arg NVIDIA_VERSION=$(NVIDIA_VERSION) \
        --build-arg HABANA_VERSION=$(HABANA_VERSION) \
        --build-arg MLX_VERSION=$(MLX_VERSION) \
        --build-arg WWDACUT_VERSION=$(WWDACUT_VERSION) \
        --build-arg RSTUDIO_VERSION=$(RSTUDIO_VERSION) \
        --build-arg PS1TAG=$(2) \
        --build-arg BUILD_TMP_DIR=$(BUILD_TMP_DIR) \
        --build-arg TARGET_ARCH=$(3) \
        --build-arg OS_VERSION=$(4) \
        --build-arg REPO_PATH=$(5) \
        --volume $(PWD):/mnt:0 \
        --tag $(2):$(DATE)_$(TIME) || { \
            echo "$(RED)Error: Failed to build $(BLUE)$(1)$(RED) variant with exit code $$?. Exiting.$(NC)"; \
            exit 1; \
        }
    @podman save $(2):$(DATE)_$(TIME) --output $(2).$(DATE)_$(TIME).tar || { \
        echo "$(RED)Error: Failed to save $(BLUE)$(1)$(RED) variant with exit code $$?. Exiting.$(NC)"; \
        exit 1; \
    }
    @echo 'wwctl image import --syncuser $(PWD)/$(2).$(DATE)_$(TIME).tar $(2).$(DATE)' >> $(INSTALL_TMP)
    @echo 'wwctl image exec $(2).$(DATE) $(DRACUT_CMD)' >> $(INSTALL_TMP)
endef
```

64k Page Kernel

Why it comes up

GraceHopper platforms prefer a kernel with 64K pages.

User space, drivers, and performance behavior can be affected by page size.

Operational considerations

Match NVIDIA driver support to the kernel configuration.

Confirm OFED, storage clients, and monitoring agents support the kernel.

Test containers and application runtimes with the page size users will actually run.

```
# Upgrade kernel to 64
RUN dnf -y --noautoremove remove \
    kernel \
    kernel-core \
    kernel-devel \
    kernel-modules \
    kernel-modules-core \
    kernel-modules-extra
```

```
RUN dnf -y install kernel-64k \
    kernel-64k-core \
    kernel-64k-devel \
    kernel-64k-modules \
    kernel-64k-modules-core \
    kernel-64k-modules-extra
```

Other Gotchas

Warewulf client overlay

Use the right wwclient binary: wwclient.x86_64 vs. wwclient.aarch64.

Boot tooling

Compile iPXE or other boot components for the target architecture.

Overlay contents

If overlays contain binaries, provide AArch64 versions.

Repos

AArch64 has its own repos, template overlays adoringly

Scheduler policy

Use partitions, features, or constraints so jobs land on the intended architecture.

Cluster Software with Multi-Arch

Build system

Use Spack, EasyBuild, or a site build pipeline to build for multiple architectures.

Module trees

Maintain separate app and module trees for x86_64 and AArch64.

User expectations

Document architecture-specific examples for CUDA, MPI, Python, and containers.

Scheduler UX

Decide whether GH200 is a separate partition, a constraint, or both.

Support policy

Make supported software combinations visible before users submit jobs.

Lets Discuss

Build system

Builds with Spack as much as possible, separate hidden folder for modules

Module trees

Maintain separate app and module trees for x86_64 and AArch64. Templated \$MODULE_PATH via /etc/profile.d

User expectations

Document architecture-specific examples for CUDA, MPI, Python, and containers.

Scheduler UX

Gracehopper is its own partition

Support policy

Make supported software combinations visible before users submit jobs.

Lets Discuss

Build system

Builds with Spack as much as possible, separate hidden folder for

Module trees

Maintain separate app and module trees for x86_64 and AArch64.

```
[root@warewulf:profile.d] PROD # wwctl overlay show scripts /etc/profile.d/spack.sh.ww
{{- if eq .Tags.cpuArch "aarch64" -}}
export MODULEPATH=/packages/aarch64/modulefiles/apps:/packages/modulefiles/spack/.arm
{{- else -}}
export MODULEPATH=/packages/modulefiles/apps:/packages/modulefiles/spack{{if eq .ClusterName "phx" }}-built{{end}}
{{- end -}}
[root@warewulf:profile.d] PROD # █
```

Questions?

Contact Info

Josh Burks
Senior HPC Systems Analyst
Arizona State University
josh.burks@asu.edu

GitHub
github.com/jeburks2/warewulf-extras

